Integration Patterns and Guidelines — Summary

Action needed: Strip the inline pattern content — keep only the table of patterns with links to child pages. Either archive or clearly label the Word doc attachments as “legacy/source” so people don't treat them as current standards.

These guidelines summarize recommended approaches and patterns for building resilient, scalable integrations, intended to guide teams toward consistent, high-quality outcomes. The emphasis is on patterns, trade-offs, and examples rather than mandates.

How to use these guidelines

- Start here for a quick overview of covered patterns
- Open a subpage for the pattern you're using to see recommendations, examples, and alternatives
- Use the “When to use / When to avoid” callouts to make pragmatic choices

Audience

- Integration engineers and architects
- Product and platform teams designing data flows
- Partners coordinating with integration services

Covered Patterns at a Glance

#	*Pattern*	*Status*	*What it's best for*
1	Asynchronous Integration	GUIDELINE AVAILABLE	High-latency backends, bursty workloads, reliable processing with queues and callbacks
2	Synchronous Integration	GUIDELINE AVAILABLE	Low-latency, small payload requests where clients need an immediate response
3	Near Real-Time Events	GUIDELINE AVAILABLE	Event-driven updates and user-facing freshness with lightweight messages
4	Batch Data Integration	GUIDELINE AVAILABLE	Larger data sets moved on a schedule with validation and monitoring
5	ETL/Data Migration	GUIDELINE AVAILABLE	Bulk transformations and historical loads using purpose-built data tools
6	Content-Based Routing	GUIDELINE AVAILABLE	Selective routing based on message attributes; prefer decoupled/event-first designs
7	Large File Handling	GUIDELINE AVAILABLE	Passing references and metadata while offloading BLOB movement to storage/services
8	Point-to-Point (Native Connectors)	GUIDELINE AVAILABLE	Direct integrations within a platform when simplicity outweighs reusability

Key Principles

- Prefer event-driven, decoupled designs for scale and resilience
- Keep synchronous payloads small and responses fast
- Use purpose-built tools for ETL and very large batch movement
- Move files and BLOBs via storage platforms; exchange references via APIs/queues
- Leverage native connectors for simple, contained, one-to-one use cases

Language note: These materials are intentionally framed as guidelines and recommendations. Where exceptions make sense, teams are encouraged to document their rationale and associated mitigations.

Appendix: Gaps to Consider Next

As we consolidate patterns and guidelines, the following topics commonly strengthen enterprise integration practices and can be addressed in subsequent iterations:

- API-led layering guidelines (System, Process, Experience) and versioning approach
- Security and data handling (OAuth2/mTLS, classification, encryption, secrets)
- Standardized error handling and resiliency (idempotency, retries, circuit breakers, rate limits)
- Observability (logging, correlation IDs, tracing, SLOs, dashboards)
- Performance and scalability guidance (load testing, capacity planning, caching)

Guideline: Large File Handling

Use integration middleware for orchestration and metadata; move large files via storage or file transfer services. This reduces latency, cost, and operational complexity.

- Prefer storage-backed transfers (e.g., S3/Azure Blob) with presigned URLs or credentials
- Minimize transformations on BLOBs; keep middleware flows lightweight
- Adopt asynchronous triggers (queues/webhooks) for reliability and scale

Alternatives and examples

- Direct system-to-system APIs using native SDKs, with middleware exchanging metadata
- SFTP/FTPS for secure, resumable transfers, with middleware orchestrating notifications
- Event messages with file references; consumers fetch from storage

When to use: large documents, images, or reports where file size spikes or throughput matters

Guideline: ETL and Data Migration

ETL workloads benefit from tools designed for bulk movement, transformations, and lineage. The integration platform is most effective orchestrating jobs and exchanging control metadata.

- Choose ETL/ELT platforms (e.g., Informatica, Talend, Glue, ADF) for heavy transforms
- Use CDC or incremental loads to reduce volume and keep sync times low
- Prefer direct DB replication/file moves for bulk; coordinate via APIs/events

Alternatives and examples

- ETL tools for heavy transforms with error/retry built in
- DB-native copy/replication for high throughput
- File-based exports/imports for simple bulk jobs

When to use: historical loads, high-volume batch jobs, complex cleansing and mapping

Guideline: Content-Based Routing

Minimize deep inspection and branching inside middleware for multi-system flows. Prefer decoupled routing through rules engines or event brokers.

- Externalize rules; keep middleware orchestration thin
- Extract routing metadata early and route on headers/properties
- Adopt event-driven distribution for scale and resilience

Alternatives and examples

- Rules engines (e.g., Drools) for flexible, auditable routing decisions
- Kafka/EventBridge topics with consumer-side filtering
- API-led orchestration using System and Process APIs

When to use: selective delivery across domains, evolving routing logic, multi-subscriber patterns

Guideline: Synchronous Interaction

Reserve synchronous requests for fast, simple lookups or updates. Bound latency, constrain payloads, and protect backends with limits.

- Keep payloads small; avoid heavy transforms on the critical path
- Apply timeouts, circuit breakers, rate limits, and caching where appropriate
- Shift slow or bursty work to asynchronous flows

Alternatives and examples

- Async request-reply via MQ/SQS for slow backends
- Webhooks/callbacks for near real-time without holding connections
- Batch APIs for bulk operations off the critical path

When to use: immediate user interactions with tight SLAs and lightweight data

Guideline: Near Real-Time Integration

Deliver timely updates with event-driven mechanisms. Treat sync endpoints as optional for read-mostly scenarios; use streams or events for continuous flow.

- Adopt pub/sub or webhooks for decoupled real-time updates
- Optimize messages to essential fields to keep latency low
- Scale workers horizontally and monitor throughput and lag

Alternatives and examples

- Kafka/SNS+SQS topics for broad distribution
- Streaming APIs (WebSocket/SSE) for dashboards
- Gateway caching for read-heavy traffic

When to use: operational dashboards, notifications, inventory or status updates

Guideline: Batch Data Integration

Schedule chunked processing with strong observability and clear recovery. Move data in bulk using storage/ETL; coordinate via middleware.

- Chunk work to control memory/CPU; parallelize thoughtfully
- Validate counts and checksums; use DLQs for failed records
- Run during off-peak windows to contain cost and contention

Alternatives and examples

- ETL jobs (Glue/ADF/Informatica) for heavy lifting
- S3/Blob storage for intermediate payloads
- MQ-based fan-out for parallel consumers

When to use: large tables/files, non-interactive workloads, predictable SLAs

Guideline: Asynchronous Integration

Use queues and events to decouple producers and consumers, improve resiliency, and smooth spikes.

- Design for idempotency; implement retries with backoff and DLQs
- Prefer metadata pointers over in-message large payloads
- Track correlation IDs across hops for traceability

Alternatives and examples

- MQ/SQS/Anypoint MQ for reliable handoff
- Event streaming for high-throughput feeds
- Callback APIs for request-completion notifications

When to use: variable-latency systems, bursty submission, long-running operations

Guideline: One-to-One Integration with Native Connectors

When requirements are contained and reusability isn't a primary goal, native connectors within a platform can simplify delivery.

- Leverage platform features (e.g., Salesforce Platform Events, Flows) for intra-platform needs
- Confirm security, logging, and upgrade paths; document configurations
- Escalate to middleware when cross-domain reuse, governance, or observability is required

Alternatives and examples

- Use middleware for shared, cross-domain flows that need governance
- Combine native connectors with light orchestration for hybrid cases

When to use: simple, well-bounded integrations where speed and simplicity matter most

---

Enterprise Integration Policies Summary

- Enterprise Integration Policies.docx
- Enterprise Integration Policy and Procedure.docx

Integration Pattern Reference

- Procedure for Asynchronous Integration.docx
- Procedure for Synchronous Pattern.docx
- Procedure for Near Real Time Integration.docx
- Procedure for Batch data integration.docx
- Procedure for ETL operations.docx
- Procedure for Content Based Routing.docx
- Procedure for large data files.docx
- Procedure for One-to-One Integration.docx
- Policy for P-2-P Integrations Using Native Connectors.docx